

Additive disturbance ladder

Start with nothing varying, switch the disturbances on one at a time, and see what each one costs.

What was run

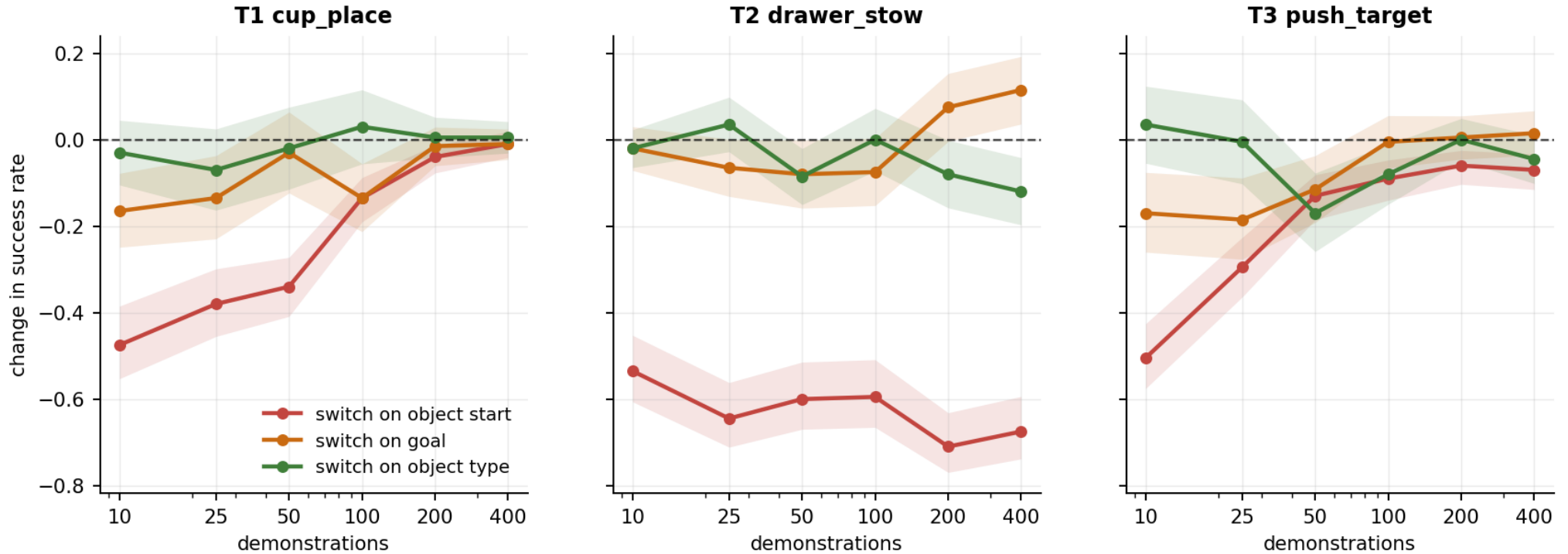
Four rungs per task. Every rung is trained at six demonstration budgets (10, 25, 50, 100, 200, 400) and scored on 200 episodes, so $4 \times 6 \times 3$ tasks = 72 policies and 14,400 episodes. Same training recipe everywhere, one seed, 80,000 steps.

rung	object start position	goal position	object type	name on disk
nothing varies	FIXED	FIXED	1 type	L0
+ object start	varies	FIXED	1 type	L1
+ goal	varies	varies	1 type	L2
+ object type	varies	varies	10 types	L3b

The three tasks are T1 cup_place (put a cup on a goal disk), T2 drawer_stow (open a drawer and stow an object in it) and T3 push_target (push a puck to a target). The cost reported for a disturbance is always **success rate with it on, minus success rate one rung below, at the same demonstration budget**. Negative means switching it on made the policy worse. All four rungs' demonstrations were generated on the same GPU, so no comparison here crosses a hardware boundary.

What each disturbance costs

What switching each disturbance on costs, given the rungs below it



Change in success rate when a disturbance is switched on, given the rungs below it. Bands are 95 % intervals. Below the dashed line it made things worse.

T1 cup_place

demonstrations	switch on object start	switch on goal	switch on object type
10	-0.475 [-0.553, -0.385]	-0.165 [-0.249, -0.078]	-0.030 [-0.105, +0.045] (not clear of zero)
25	-0.380 [-0.455, -0.299]	-0.135 [-0.229, -0.037]	-0.070 [-0.163, +0.025] (not clear of zero)
50	-0.340 [-0.409, -0.272]	-0.030 [-0.123, +0.064] (not clear of zero)	-0.020 [-0.115, +0.075] (not clear of zero)
100	-0.135 [-0.190, -0.088]	-0.135 [-0.213, -0.056]	+0.030 [-0.056, +0.115] (not clear of zero)
200	-0.040 [-0.077, -0.013]	-0.015 [-0.060, +0.029] (not clear of zero)	+0.005 [-0.041, +0.052] (not clear of zero)
400	-0.010 [-0.041, +0.018] (not clear of zero)	-0.010 [-0.046, +0.024] (not clear of zero)	+0.005 [-0.031, +0.042] (not clear of zero)

T2 drawer_stow

demonstrations	switch on object start	switch on goal	switch on object type
10	-0.535 [-0.606, -0.453]	-0.020 [-0.071, +0.030] (not clear of zero)	-0.020 [-0.065, +0.023] (not clear of zero)
25	-0.645 [-0.711, -0.562]	-0.065 [-0.131, +0.001] (not clear of zero)	+0.035 [-0.028, +0.098] (not clear of zero)
50	-0.600 [-0.670, -0.515]	-0.080 [-0.158, -0.001]	-0.085 [-0.150, -0.021]
100	-0.595 [-0.665, -0.509]	-0.075 [-0.152, +0.003] (not clear of zero)	+0.000 [-0.072, +0.072] (not clear of zero)
200	-0.710 [-0.769, -0.632]	+0.075 [-0.004, +0.153] (not clear of zero)	-0.080 [-0.158, -0.001]
400	-0.675 [-0.738, -0.594]	+0.115 [+0.036, +0.192]	-0.120 [-0.197, -0.042]

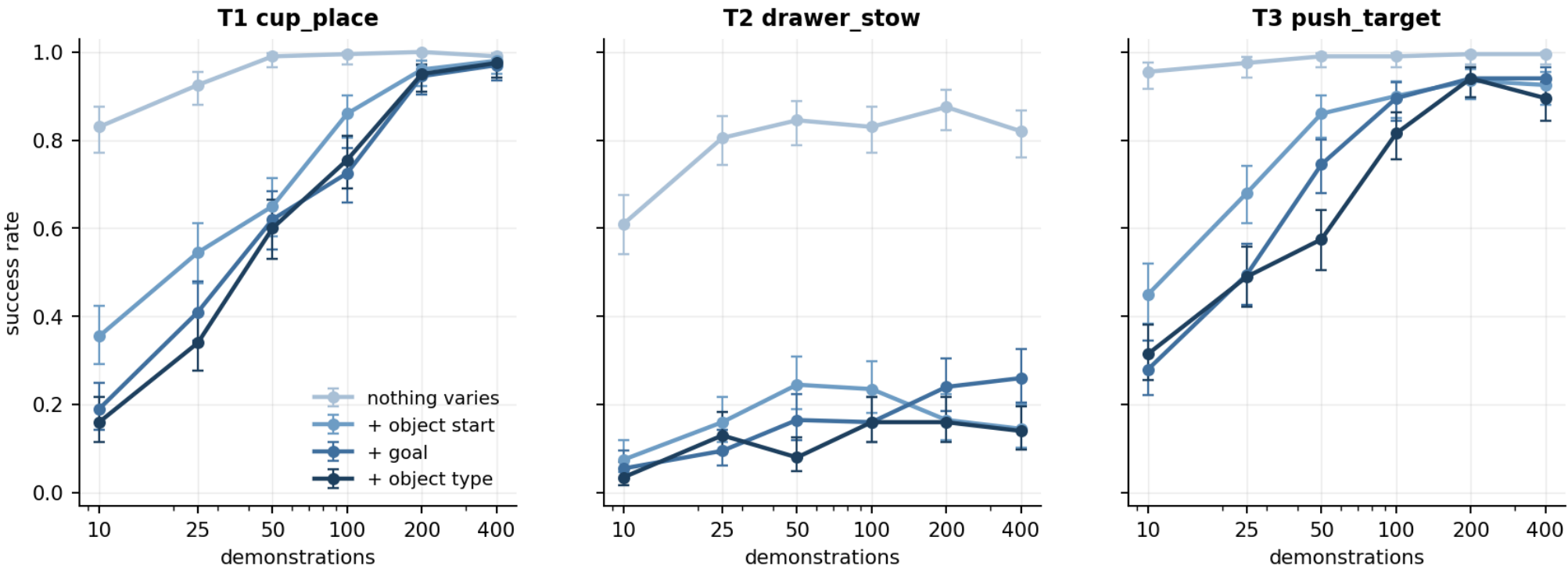
T3 push_target

demonstrations	switch on object start	switch on goal	switch on object type
10	-0.505 [-0.576, -0.426]	-0.170 [-0.260, -0.076]	+0.035 [-0.054, +0.124] (not clear of zero)
25	-0.295 [-0.364, -0.226]	-0.185 [-0.277, -0.089]	-0.005 [-0.102, +0.092] (not clear of zero)
50	-0.130 [-0.185, -0.081]	-0.115 [-0.192, -0.037]	-0.170 [-0.259, -0.077]
100	-0.090 [-0.140, -0.047]	-0.005 [-0.066, +0.056] (not clear of zero)	-0.080 [-0.149, -0.011]
200	-0.060 [-0.103, -0.025]	+0.005 [-0.045, +0.055] (not clear of zero)	+0.000 [-0.049, +0.049] (not clear of zero)
400	-0.070 [-0.115, -0.033]	+0.015 [-0.036, +0.067] (not clear of zero)	-0.045 [-0.101, +0.010] (not clear of zero)

Read each budget on its own line. The same disturbance can cost a lot at 25 demonstrations and nothing at 400.

The success rates behind those costs

Success rate at each rung of the ladder



Success rate at each rung. Error bars are 95 % intervals on 200 episodes.

T1 cup_place

demonstrations	nothing varies	+ object start	+ goal	+ object type
10	0.830 (166/200)	0.355 (71/200)	0.190 (38/200)	0.160 (32/200)
25	0.925 (185/200)	0.545 (109/200)	0.410 (82/200)	0.340 (68/200)
50	0.990 (198/200)	0.650 (130/200)	0.620 (124/200)	0.600 (120/200)
100	0.995 (199/200)	0.860 (172/200)	0.725 (145/200)	0.755 (151/200)
200	1.000 (200/200)	0.960 (192/200)	0.945 (189/200)	0.950 (190/200)
400	0.990 (198/200)	0.980 (196/200)	0.970 (194/200)	0.975 (195/200)

T2 drawer_stow

demonstrations	nothing varies	+ object start	+ goal	+ object type
10	0.610 (122/200)	0.075 (15/200)	0.055 (11/200)	0.035 (7/200)
25	0.805 (161/200)	0.160 (32/200)	0.095 (19/200)	0.130 (26/200)
50	0.845 (169/200)	0.245 (49/200)	0.165 (33/200)	0.080 (16/200)
100	0.830 (166/200)	0.235 (47/200)	0.160 (32/200)	0.160 (32/200)
200	0.875 (175/200)	0.165 (33/200)	0.240 (48/200)	0.160 (32/200)
400	0.820 (164/200)	0.145 (29/200)	0.260 (52/200)	0.140 (28/200)

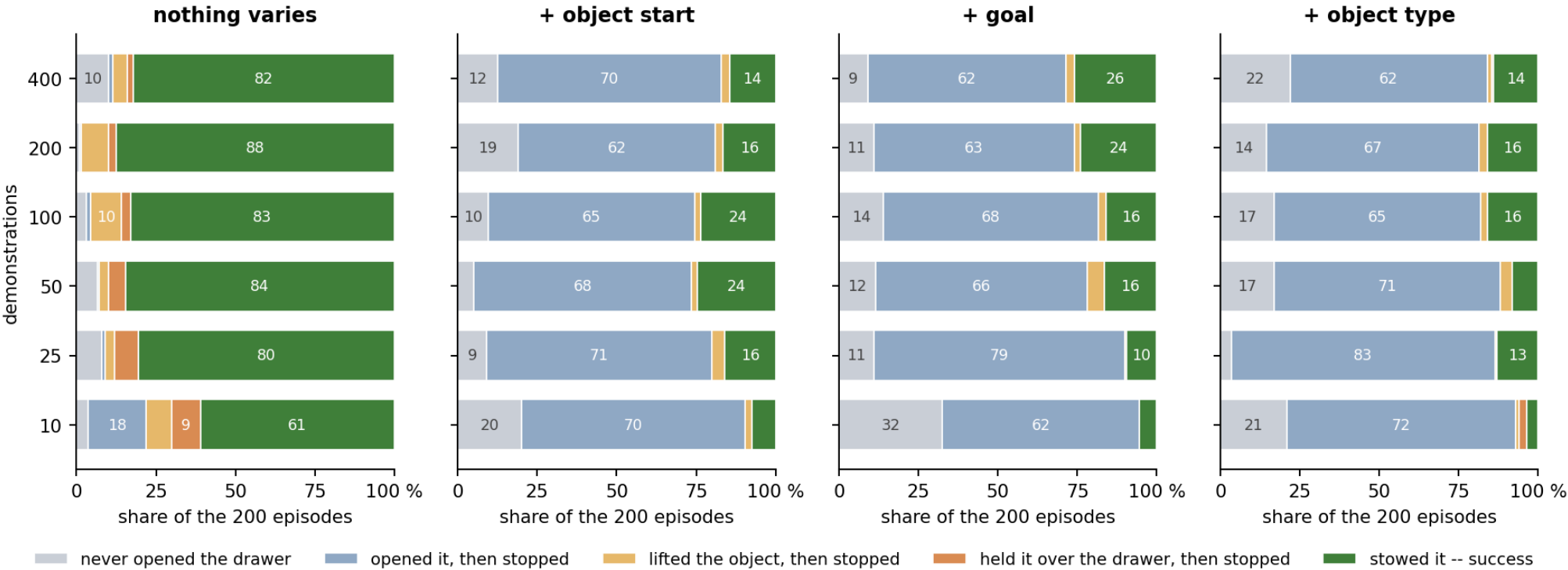
T3 push_target

demonstrations	nothing varies	+ object start	+ goal	+ object type
10	0.955 (191/200)	0.450 (90/200)	0.280 (56/200)	0.315 (63/200)
25	0.975 (195/200)	0.680 (136/200)	0.495 (99/200)	0.490 (98/200)
50	0.990 (198/200)	0.860 (172/200)	0.745 (149/200)	0.575 (115/200)
100	0.990 (198/200)	0.900 (180/200)	0.895 (179/200)	0.815 (163/200)
200	0.995 (199/200)	0.935 (187/200)	0.940 (188/200)	0.940 (188/200)
400	0.995 (199/200)	0.925 (185/200)	0.940 (188/200)	0.895 (179/200)

Where T2 policies got stuck

T2 is a four-part task: open the drawer, pick up the object, carry it over the open drawer, drop it in. Every episode records how far it got, so a failure can be placed instead of just counted. Each bar below is 200 episodes split by the furthest point reached, so the five shares add to 100.

T2 drawer_stow: how far each episode got before it stopped



How far each episode got before it stopped. Numbers are percentages of 200 episodes.

rung	demos	never opened	opened, stopped	lifted, stopped	held over, stopped	stowed it
nothing varies	10	7	37	16	18	122
nothing varies	25	16	2	6	15	161
nothing varies	50	13	1	6	11	169
nothing varies	100	6	3	19	6	166
nothing varies	200	2	1	17	5	175
nothing varies	400	20	3	9	4	164

rung	demos	never opened	opened, stopped	lifted, stopped	held over, stopped	stowed it
+ object start	10	40	141	4	0	15
+ object start	25	18	142	8	0	32
+ object start	50	10	137	4	0	49
+ object start	100	19	130	4	0	47
+ object start	200	38	124	5	0	33
+ object start	400	25	141	5	0	29
+ goal	10	65	124	0	0	11
+ goal	25	22	158	1	0	19
+ goal	50	23	133	11	0	33
+ goal	100	28	135	5	0	32
+ goal	200	22	126	4	0	48
+ goal	400	18	125	5	0	52
+ object type	10	42	144	2	5	7
+ object type	25	7	166	1	0	26
+ object type	50	34	142	8	0	16
+ object type	100	34	130	4	0	32
+ object type	200	29	134	5	0	32
+ object type	400	44	124	3	1	28

Counts out of 200 episodes.

What could be wrong with these numbers

#	issue	what was measured
1	Each rung is measured on top of the rungs below it.	The cost of switching on the goal is measured with the object start already varying, and the cost of object type with both of the others already on. A different order would give different numbers. This report measures one order, the one the ladder defines.
2	One training seed.	Every policy is seed 0. The intervals here cover episode sampling only; they contain no training-run variance.
3	Repeating an evaluation does not give the identical episodes back.	Re-running one 20-episode slice with the same seed and checkpoint reproduced the slice total but flipped two individual episodes. Expect about +/-1 success per 20 episodes of jitter.
4	Do not average a change across demonstration budgets.	The six budgets are six different policies. A test for a shared success rate is rejected in all 18 rung-task pairs, so an average over budgets describes the mix of budgets chosen, not the rung.
5	These numbers replace an earlier scoring of the same policies.	The success counter used to read a flag that stayed latched across episode boundaries, which credited successes that never happened. All 108 cells were re-scored after the fix and this report uses only the corrected results. Older surfaces in the repository for the same cells are superseded.

What would fix them

#	next step
1	Train more than one seed on whichever rungs matter.
2	Measure the ladder in a second order, so the conditioning in caveat 1 can be quantified.
3	Record the evaluator version in the results ledger, and which git revision each run used.
4	Measure whether a second warm-up batch moves the scored success rate.
5	Mark the superseded surface files in the repository so they cannot be picked up by mistake.

Every number here comes from `src/cog/analysis/ladder.py`, over the pooled result files in `results/`. The tables are also plain CSV in `paper/ladder_report/tables/`.